

✓ linear regression practical implementaion

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
```

```
from sklearn.datasets import fetch_openml
boston=fetch_openml(name= 'boston',version=1,as_frame = True,parser='pandas')
```

boston

```
{'data':      CRIM      ZN  INDUS  CHAS    NOX     RM   AGE     DIS  RAD    TAX  \
0   0.00632  18.0    2.31    0  0.538  6.575  65.2  4.0900  1  296.0
1   0.02731   0.0    7.07    0  0.469  6.421  78.9  4.9671  2  242.0
2   0.02729   0.0    7.07    0  0.469  7.185  61.1  4.9671  2  242.0
3   0.03237   0.0    2.18    0  0.458  6.998  45.8  6.0622  3  222.0
4   0.06905   0.0    2.18    0  0.458  7.147  54.2  6.0622  3  222.0
..      ...     ...     ...     ...     ...     ...     ...     ...  ..  ...
501  0.06263   0.0   11.93    0  0.573  6.593  69.1  2.4786  1  273.0
502  0.04527   0.0   11.93    0  0.573  6.120  76.7  2.2875  1  273.0
503  0.06076   0.0   11.93    0  0.573  6.976  91.0  2.1675  1  273.0
504  0.10959   0.0   11.93    0  0.573  6.794  89.3  2.3889  1  273.0
505  0.04741   0.0   11.93    0  0.573  6.030  80.8  2.5050  1  273.0

      PTRATIO      B  LSTAT
0         15.3  396.90   4.98
1         17.8  396.90   9.14
2         17.8  392.83   4.03
3         18.7  394.63   2.94
4         18.7  396.90   5.33
..      ...     ...     ...
501        21.0  391.99   9.67
502        21.0  396.90   9.08
503        21.0  396.90   5.64
504        21.0  393.45   6.48
505        21.0  396.90   7.88

[506 rows x 13 columns],
'target': 0      24.0
1      21.6
2      34.7
3      33.4
4      36.2
..      ...
501     22.4
502     20.6
503     23.9
504     22.0
505     11.9
Name: MEDV, Length: 506, dtype: float64,
'frame':      CRIM      ZN  INDUS  CHAS    NOX     RM   AGE     DIS  RAD    TAX  \
0   0.00632  18.0    2.31    0  0.538  6.575  65.2  4.0900  1  296.0
1   0.02731   0.0    7.07    0  0.469  6.421  78.9  4.9671  2  242.0
2   0.02729   0.0    7.07    0  0.469  7.185  61.1  4.9671  2  242.0
3   0.03237   0.0    2.18    0  0.458  6.998  45.8  6.0622  3  222.0
4   0.06905   0.0    2.18    0  0.458  7.147  54.2  6.0622  3  222.0
..      ...     ...     ...     ...     ...     ...     ...     ...  ..  ...
```

```

501 0.06263 0.0 11.93 0 0.573 6.593 69.1 2.4786 1 273.0
502 0.04527 0.0 11.93 0 0.573 6.120 76.7 2.2875 1 273.0
503 0.06076 0.0 11.93 0 0.573 6.976 91.0 2.1675 1 273.0
504 0.10959 0.0 11.93 0 0.573 6.794 89.3 2.3889 1 273.0
505 0.04741 0.0 11.93 0 0.573 6.030 80.8 2.5050 1 273.0

```

```

PTRATIO B LSTAT MEDV
0 15.3 396.90 4.98 24.0
1 17.8 396.90 9.14 21.6
2 17.8 392.83 4.03 34.7
3 18.7 394.63 2.94 33.4
4 18.7 396.90 5.33 36.2

```

```
df=boston.frame
```

```
df.head()
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296.0	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242.0	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242.0	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222.0	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222.0	18.7	396.90	5.33	36.2

```
df['price'] = boston.target
df.head()
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV	price
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296.0	15.3	396.90	4.98	24.0	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242.0	17.8	396.90	9.14	21.6	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242.0	17.8	392.83	4.03	34.7	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222.0	18.7	394.63	2.94	33.4	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222.0	18.7	396.90	5.33	36.2	36.2

✓ dividing the data set into dependent and independent

```
x = df.iloc[:, :-1]
```

```
x.head()
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296.0	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242.0	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242.0	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222.0	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222.0	18.7	396.90	5.33	36.2

```
y= df.iloc[:,-1]
```

```
y.head()
```

```
0    24.0
1    21.6
2    34.7
3    33.4
4    36.2
Name: price, dtype: float64
```

✓ linear regression

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score
x = x.astype(float)
y = y.astype(float)

lin_reg=LinearRegression()
mse = cross_val_score(lin_reg,x,y,scoring = 'neg_mean_s
```